EZRA Take Home Assignment

Question1:

Part 1: Top 15 TestCases in descending order by priority covering testcases from various testing strategies point of view (Functional, Security, Accessibility, Localization etc). More details about the testcases are provided in the attached excel sheet.

1. **Verify that the user can successfully book the scanning appointment with valid payment using https protocol and receive a confirmation and verify details are created correctly in User Facing Portal**
2. **Verify that a scanning appointment is not created when the payment is failed or declined**
3. **Verify that the system prevents duplicate charges or duplicate bookings when the payment is submitted multiple times or the page is refreshed**
4. Verify that the selected appointment time slot is locked only after a successful payment to prevent overbooking
5. Verify that when multiple users attempt to book the same time slot, only the first successful payment secures the appointment
6. Verify that the Continue button state and validation messages are properly announced by screen readers
7. Verify that the system prevents users from proceeding when an invalid or future date of birth is entered
8. Verify that all required fields have proper programmatic labels and required-state indicators for accessibility
9. Verify that users cannot view or select calendar dates or time slots without first selecting a location
10. Verify that the date format displayed in the calendar and selected date respects the user's locale settings
11. Verify that users cannot proceed without selecting an appointment time
12. Verify that the displayed time zone and time format (12-hour or 24-hour) are localized correctly based on user's locale
13. Verify that payment data is securely handled and that sensitive information is masked and not exposed in the UI or URLs
14. Verify that the payment form supports complete keyboard navigation with a visible and logical focus order
15. Verify that the currency symbol and payment amount are displayed correctly according to the user's locale
16. Verify every negative tests like creating same member user, trying to book an appointment (with an endpoint) using already booked time slot etc etc

Part 2:

- TC#1 is undoubtedly highest priority as it is primary user journey and validates core end-to-end business flow (UI → backend business logic → Payment gateway) that Ezra depends on for revenue generation and customer fulfillment. It confirms that all key components (scan selection, scheduling, payment processing, booking creation, and confirmation) work together correctly and meets the acceptance criteria. On the other hand, if there is a failure that results in reduced bookings/earnings, damage to the Ezra's brand value, unsatisfied customers, and

workforce reduction. This testcase must be part of automated flame test that would give immediate feedback should there an issue with the flow.

- TC#2 is second highest priority as the test ensures data integrity and financial correctness by validating that bookings are created only after a successful payment since creating Scan appointments with invalid payment does not meet the acceptance criteria and loss to company resources. This can happen due to real-world scenarios like declined cards, insufficient funds, expired cards, network issues etc, and the Ezra's booking system must handle them graciously. On the other hand, if there is a failure in this workflow then time slots will be reserved incorrectly and manual intervention/customer disputes may occur. So, to avoid, revenue leakage, scheduling errors, and customer dissatisfaction, making it one of the most important negative-path scenarios. Some users might report to BBB which diminishes company reputation and brand value.

- TC#3 is next priority testcase as it commonly occurs during payment processing or lack of warning message on UI (eg: Please do not refresh or click back button on the browser while the processing is in progress). For example, user actions like double-clicking the submit button, refreshing the page, or experiencing latency during payment usually results in multiple charges to credit card resulting in loss of customer trust and ultimately reducing customer churn even though refunds happen after the fact. This is sometimes may lead to civil suites on the company for illegally charging multiple times (or holding the card user's funds).

Question2:

Part 1: Verify that a member cannot access another member's Medical Questionnaire by manipulating the "Begin Medical Questionnaire" URL parameters (encounterId). This ensures server-side authorization enforcing authenticated member can only access their own medical questionnaire even if they guess or obtain another encounterId.

https://myezra-staging.ezra.com/medical-questionnaire?flow=medical-questionnaire&direct=true&clearData=true&extraData={%22encounterId%22:%2285fce212-9a2d-405e-8bc5-5d722a62c307%22}

Precondition:

Two valid member accounts exist (eg: Member A and Member B). Each member has an appointment that generates a unique encounterId used in the Medical Questionnaire hyperlink.

Test Data:

encounterId_A = encounter UUID belonging to Member A

encounterId_B = encounter UUID belonging to Member B

Test Steps:

1. Login as Member A in Browser Session 1

2. Navigate to Begin Medical Questionnaire and confirm you reach the questionnaire start screen (e.g., “General Questions”).

3. Copy the current questionnaire URL and replace the encounterId value with encounterId_B (Member B’s encounter).

4. Paste the modified URL into the address bar while still authenticated as Member A and refresh the page.

Expected Results (Pass Criteria):

1. The application must not display Member B’s medical questionnaire screens or pre-filled data

2. Backend responds with 403 Forbidden (ideally) or 404 Not Found (acceptable), and does not return PHI/medical questionnaire content for encounterId_B to Member A.

3. Redirect to dashboard/login or a generic page that shows “Access denied” state (Ensure, no sensitive identifiers leaked in error messages)

4. On Database side, log telemetry record with unauthorized access attempt. From Security testing prospective, it is known as OWASP Top 10 authorization failure and should send email to Security Team

Note: In the questionnaire, there is an option to fill details for other member. If the user chooses to fill the details for another member (eg: Spouse or parent) then ensure that it should give a message to logout and relogin using other member's credentials. Better yet, if other applicant is minor then the details are stored under a different encountered and linked to main member (out of scope in the product). This can be second integration testcase related to privacy.

Note: In my last organization, I found similar privacy/security bug. User1 created Journal_ID_1. User2 created JournalID_2. Invoke getJournalDetails endpoint for JournalID_1. Then replace Journal_ID_1 with JournalID_2. In this case, due to server side security lapse, User1 could access journals created by other users.

Part 2

Here are the main endpoints triggered as part of Begin Questionnaire.

Method	**endpoint**
POST	/individuals/member/connect/token
GET and PATCH	/individuals/api/members
GET	/diagnostics/api/medicalfiles/identification

GET	/ diagnostics/api/medicaldata/forms/mq/submissions/<package_id>/detail
GET and POST	/diagnostics/api/medicaldata/forms/mq/submissions/<id>/data
POST	/diagnostics/api/medicaldata/forms/mq/submissions/<id>/complete
GET	/individuals/api/members/{id}/appointments
GET	/individuals/api/members/{id}/tasks
GET	/individuals/api/members/{id}/medical-questionnaire/{eid}
GET	/individuals/api/members/{id}/medical-questionnaire/ {eid}/responses

Part 3:

Background: I have worked at 3 MedTech companies (i.e **handled PII/PHI/HIPPA data**) and also worked at an electrical software company (i.e A software which integrates with electrical grid and tested the endpoints keeping **national security** in mind). With this experience, here are my thought process.

Strategies to Perform Security Focused QA Testing:

1. Penetration Testing and Vulnerability Scanning

   This is the most critical step, involving authorized simulated attacks to identify security vulnerabilities.

Test Strategy	Focus Area	Best Practice
Vulnerability Scanning	Automated identification of known weaknesses in code, libraries, and infrastructure configurations.	Run scans on all 100+ endpoints regularly (e.g., nightly or on every build) to catch common flaws like SQL injection, XSS, and misconfigurations.
Penetration Testing	Manual, in-depth exploration of vulnerabilities, simulating a real-world attacker.	Conduct annual, comprehensive penetration tests by a third party, focusing on data flow, authentication, and authorization logic across the entire suite of 100+ endpoints.
API Security Testing	Specific to the endpoint layer, excessive data exposure, and improper asset management.	Use tools like OWASP ZAP or Burp Suite to automate security checks for every API request and response, ensuring PII/PHI is never exposed unnecessarily.

2. Access Control and Authentication Testing

HIPAA mandates that access to Protected Health Information (PHI) be restricted to the minimum necessary for a user's role.

Test Area	QA Testing Requirement
Role-Based Access Control (RBAC)	Verify that a user with Role A cannot access data or invoke an endpoint restricted to Role B. This requires a matrix of all 100+ endpoints mapped to every user role.
Authentication Protocols	Test for strong password policies, multi-factor authentication (MFA) enforcement, and secure session management (e.g., proper session termination and short, secure session timeouts).
Input Validation	Ensure all data inputs are strictly validated to prevent injection attacks, especially in parameters that query or modify sensitive data.

3. Data Integrity and Encryption Validation

All sensitive data must be protected both when it is stored (at rest) and when it is transmitted (in transit).

State of the Data	QA Testing Requirement
Data in Transit	Verify that all endpoint communication uses TLS 1.2 or higher and that unencrypted HTTP requests are rejected. Test for man-in-the-middle vulnerabilities.
Data at Rest	Confirm that all databases and storage mechanisms containing PII/PHI use strong encryption protocol. Test the decryption/re-encryption process during data access. Data Recovery processes are in place (read: backing up data from time to time).
Data Integrity	Validate that the data is not corrupted or altered during transfer, especially across multiple endpoints (eg: ascii chars, special chars, international chars etc). Use checksums or hashing to verify consistency.

4. Audit Trails and Logging Verification

Comprehensive, tamper-proof logging is a core HIPAA requirement for accountability.

Logging Area	QA Testing Requirement

Event Logging	Verify that every access, modification, or deletion of PHI/PII is logged, including the user ID, timestamp, and the specific data accessed. Test for man-in-the-middle vulnerabilities.
Tamper-Proofing	Test the system's ability to detect and alert on any unauthorized attempts to modify or delete audit logs. Test the decryption/re-encryption process during data access.
Alerting	Validate that security events (e.g., multiple failed login attempts, unauthorized access attempts) trigger immediate, high-priority alerts to the security team.

Most important: The most significant security risk in QA/Dev is the use of real production data. Never use real PII or PHI in non-production environments or during reproduction of customer issues in QA/Dev Env. The choice between synthetic data and data masking is a key strategic decision. If QA/Dev team can not mask the data then the reproduction of customer issue should be done using a specialized environment (eg: hotfix env) located in a secured zone where there are limited actions can be done on this env (for example restricted internet, copy, download etc actions are disabled).

Tradeoffs and Potential Risks

Implementing a high-security QA strategy involves several tradeoffs and risks that must be managed:

Tradeoffs

1. Cost and Time vs. Security:
    - Tradeoff**:** Security-focused QA, especially penetration testing and synthetic data generation, is significantly more expensive and time-consuming than traditional functional testing.
    - Mitigation**:** Automate as much security testing as possible (e.g., API security scans) to reserve manual effort for high-risk areas.
2. Data Realism vs. Compliance:
    - Tradeoff**:** Synthetic data, while fully compliant, may not perfectly replicate the complex data distributions or rare edge cases found in production, potentially leading to undiscovered functional bugs.
    - Mitigation**:** Invest in advanced synthetic data tools that can accurately model data relationships and use a small, highly-controlled, and strictly isolated masked dataset for final pre-production validation of critical flows or use hotfix envs as mentioned above
3. Scope vs. Depth:
    - Tradeoff**:** With 100+ endpoints, testing every single endpoint with the same depth is impractical.
    - Mitigation**:** Implement a **risk-based testing strategy**. Prioritize the endpoints that handle the most sensitive data (e.g., creation, update, or deletion of PHI) or those that are publicly exposed, dedicating the deepest security testing to them.

Potential Risks

1. Inconsistent Security Across Endpoints: A major risk with a large number of endpoints is that security standards or testing coverage may vary. A single weak endpoint can compromise the

entire system.
    - Mitigation**:** Enforce a centralized, standardized security testing framework and use automated checks to ensure all endpoints adhere to the same security headers, authentication, and logging policies.
2. Re-identification Risk (If using Masked Data): Even if data is masked, combining multiple masked datasets or using advanced statistical techniques can sometimes reverse the masking process, exposing the original PHI.
    - Mitigation**:** If masking is used, ensure the process is **irreversible** and that the masked data is never combined with any other potentially identifying information.
3. Compliance Changes: Security and compliance requirements (HIPAA etc.) evolve constantly. A system that is compliant today may not be tomorrow.
    - Mitigation**:** Establish a continuous compliance monitoring program and integrate regulatory updates into the QA and security testing cycles at least quarterly. Basically, Audit, Security, and Software Teams should be up-to-date on latest protocols and trainings.

By adopting these security-focused QA methodologies and carefully managing the tradeoffs, QA can build a robust defense for sensitive data endpoints, ensuring both security and regulatory compliance.

###